1. Explain the major features of Object Oriented Programming?

Ans OOPs is a paradigm that organizes software design around data, objects rather than function or logics. There major feature are :-

→ Encapsulation:
- Encapsulation binds together data (variable) and functions (methods) that operate on data.
- It ensures data hiding by restricting direct access using access modifiers like public, private & protected.

→ Abstraction:
- Abstraction is the process of hiding complex implementation details and only showing essential details
- Java achieves it using abstract classes & interfaces

→ Inheritance:
- Inheritance allows a class (child / subclass) to inherit properties and method from another class (parent / superclass)
- It promotes code reusability and modularity

→ Polymorphism:
- Polymorphism means "many forms". It allows one interface to be used for a general class
- Types
  1. Compile time (Static): method overloading
  2. Run time (Dynamic): method overriding.

Q2) Compare Java Runtime Environment and Java Virtual Environment.

	JVM	JRE
Defination	A virtual machine that interprets and executes Java bytecode	A software package that provides libraries, JVM and other tools to run Java program
Responsible	Executes the bytecode line by line, ensures portability and security	Creates runtime environment to execute Java applications.
Platform	Platform independent	Platform dependent
Interaction	Work in background	Act as user facing environ

Q3) How are object implemented in Java?

→ Object in Java are implemented using the new keyword which allocates memory dynamically.

→ Each object has State (defined by attributes), behaviour (by method) and identity (unique reference).

```
public class Car {
    int speed;

    void drive() {
        System.out.println("The speed is");
    }

    public static void main(String args[]) {
        Car c = new Car(); // Obj creation
        c1.speed = 100; // implementation.
        c1.drive();
    }
}
```

Q4) Whats are the limitation of static members?

Ans In Java, static member belong to the class rather than any specific instance of the class.

Limitation

1. No access to Instance Variable / Method directly
   static members can not access non static members directly without object

2. Lack of Polymorphism.
   Static method can not be overriden

3. Global State Risk:
   Since static fields are shared across all instance any change by one instance affects all other

Q5) What do you mean by constructor? How many types of constructor are in java? Explain with suitable example.

Ans A constructor is a special method that is invoked automatically when an object is created. It initializes object properties and has no return type.

→ It has same name as of class.
→ If constructor is not defined, default constructor is used.

⇒ Type of Constructor:

- Default Constructor:

→ Implicitly created by JAVA if no constructor defined.

```
Ex: class Student(){
        Student(){
            System.out.println("Hello");
        }
    }
```

- Parameterized

→ It may take parameters to accept and initialize instance variable

```
Ex: class Student(){
        String name;
        int age;
        Student(String n, int a){
            name = n;
            age = a;
        }
```

Q6) Describe in detail about the type of inheritance.

Ans Inheritance in Java allows a class to inherit the fields & methods of another class.

Type of Inheritance in Java :

1 Single Inheritance : A class inherits from 1 superclass.

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}
```

2 Multilevel Inheritance : A class inherits from a subclass, forming a chain.

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Bark");
    }
}
class Puppy extends Dog {
    void weep() {
        System.out.println("Weeping");
    }
}
```

3. Hierarchical Inheritance:
- Multiple classes inherit from a single superclass.

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}
class Dog extends Animal {}
class Cat extends Animal {}
```

4. Hybrid Inheritance:
- Combination of more than one type of inheritance
- Achievable through interface, not directly

Q7) Which is the alternative approach to implement multiple inheritance in Java Explain with Example?

Ans To avoid ambiguity caused by the diamond problem, Java does not support multiple inheritance with classes, Thread, Interfaces allows a class to implement multiple behaviour.

Hence: Java uses interface to achieve Multiple Inheritance.

```
interface Printable {
    void print();
}

interface Showable {
    void show();
}

class A implements Printable, Showable {
    public void print() {
        System.out.println("Print");
    }

    public void Show() {
        System.out.println("Show");
    }
}

public class Demo {
    public static void main(String args[]) {
        A obj = new A();
        obj.print();
        obj.show();
    }
}
```

Page No.:

Q8) What are packages? How are they created & accessed in Java.

Ans A package in Java is a namespace that organizes classes and interface. It prevents naming conflicts and makes code modular and maintainability.

Types:
→ Build in package : java.util, java.io, etc
→ User defined package : Created by developer

Creating a package :
→ They are declared using keyword "package".

```
package mypackage;
public class Message {
    public void display () {
        System.out.println("Hello World");
    }
}
```

Accessing a Package :

```
import mypackage.Message;
public class Test {
    public static void main (String args[]) {
        Message m = new Message ();
        m.display ();
    }
}
```

✓The Good Paper

Q9) Explain Polymorphism and types of polymorphism with ex.

Ans.

Polymorphism refers to the ability of a single interface or method to behave differently based on the object or context. Promote flexibility and reusability in code.

Type of Polymorphism:

→ Compile-Time Polymorphism (Static Binding):

- Achieved using method overloading.
- The method call is resolved at compile time

Example:

```
Class Adder {
    int add (int a, int b) {
        return a+b;
    }
    double add (double a, double b)
    { return a+b; }
}
```

→ Runtime Polymorphism (Dynamic Binding):

- Achieved through method overriding.
- Determined during runtime based on object type

```
Class Animal {
    void sound() {
        System.out.println("Animal");
    }
```

```
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Test {
    public static void main(String args[]) {
        Animal a = new Dog();
        a.sound();
    }
}
```

10.) Explain Interface & abstract class with explan

Abstract Class:

→ A class declared with the abstract keyword.
→ Can contain abstract and non abstract method
→ Support partial abstraction
→ Can have constructors, static methods, etc

Example:

```
abstract class Shape {
    abstract void draw();
    void message() {
        System.out.println("Shape");
    }
}
class Circle extends Shape {
    void draw() {
        System.out.println("Drawn");
    }
}
```

Interface:

- An Interface is a blueprint of a class.
- It is a reference type, similar to a class, that can contain only abstract method, default method and constants.
- They are used to achieve abstraction and to implement multiple inheritance in Java

Example:

```
interface Drawable {
    void draw();
}

class Circle implements Drawable {
    public void draw() {
        System.out.println("Drawing Circle");
    }
}
```